

Хуки React

Введение в хуки React

Хуки — это функции, которые позволяют использовать состояние и другие возможности React без написания классов. Они были представлены в React 16.8 и стали стандартным способом управления состоянием и побочными эффектами в функциональных компонентах.

useState

Хук `useState` позволяет добавить состояние в функциональный компонент.

```
import React, { useState } from 'react';

function Counter() {
  // Объявляем новую переменную состояния "count"
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Вы нажали {count} раз</p>
      <button onClick={() => setCount(count + 1)}>
        Нажми меня
      </button>
    </div>
  );
}
```

`useState` возвращает пару значений: текущее состояние и функцию, которая его обновляет.

useEffect

Хук `useEffect` позволяет выполнять побочные эффекты в функциональных компонентах. Он заменяет методы жизненного цикла `componentDidMount`, `componentDidUpdate` и `componentWillUnmount` в классовых компонентах.

```
import React, { useState, useEffect } from 'react';

function Example() {
  const [count, setCount] = useState(0);

  // Аналогично componentDidMount и componentDidUpdate
  useEffect(() => {
    // Обновляем заголовок документа, используя API браузера
    document.title = `Вы нажали ${count} раз`;
  });
}
```

```

return (
  <div>
    <p>Вы нажали {count} раз</p>
    <button onClick={() => setCount(count + 1)}>
      Нажми меня
    </button>
  </div>
);
}

```

Очистка эффектов

Некоторые эффекты требуют очистки, например, подписки или таймеры. Для этого функция, переданная в `useEffect`, может вернуть функцию очистки.

```

useEffect(() => {
  const subscription = props.source.subscribe();
  return () => {
    // Очистка подписки перед размонтированием компонента
    subscription.unsubscribe();
  };
});

```

Зависимости эффектов

Можно оптимизировать производительность, пропуская эффекты, если определенные значения не изменились между повторными рендерами. Для этого передайте массив зависимостей в качестве второго аргумента `useEffect`.

```

useEffect(() => {
  document.title = `Вы нажали ${count} раз`;
}, [count]); // Повторно запускать эффект только если count изменился

```

useContext

Хук `useContext` позволяет подписаться на контекст React без введения вложенности.

```

import React, { useContext } from 'react';

const ThemeContext = React.createContext('light');

function ThemedButton() {
  const theme = useContext(ThemeContext);

```

```
return <button className={theme}>Я стилизован темой из контекста</button>;
}
```

useReducer

Хук `useReducer` представляет альтернативу `useState` для управления сложным состоянием.

```
import React, { useReducer } from 'react';

function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return {count: state.count + 1};
    case 'decrement':
      return {count: state.count - 1};
    default:
      throw new Error();
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });
  return (
    <>
      Счетчик: {state.count}
      <button onClick={() => dispatch({type: 'decrement'})}>-</button>
      <button onClick={() => dispatch({type: 'increment'})}>+</button>
    </>
  );
}
```

useCallback

Хук `useCallback` возвращает мемоизированную версию колбэка, которая изменяется только если изменяются значения в массиве зависимостей.

```
import React, { useState, useCallback } from 'react';

function ParentComponent() {
  const [count, setCount] = useState(0);

  // useCallback предотвращает ненужные повторные рендеры дочерних компонентов
  const handleClick = useCallback(() => {
    setCount(c => c + 1);
  }, []); // Пустой массив зависимостей означает, что функция создается только
  один раз

  return (
```

```

    <div>
      <ChildComponent onClick={handleClick} />
      <p>Счетчик: {count}</p>
    </div>
  );
}

```

useMemo

Хук `useMemo` возвращает мемоизированное значение, которое пересчитывается только при изменении одной из зависимостей.

```

import React, { useMemo } from 'react';

function ExpensiveCalculation({ a, b }) {
  // Результат вычисления будет сохранен и повторно использован,
  // если a и b не изменились
  const result = useMemo(() => {
    console.log('Выполняется сложное вычисление...');
    return a * b * 1000;
  }, [a, b]);

  return <div>Результат: {result}</div>;
}

```

useRef

Хук `useRef` возвращает изменяемый ref-объект, свойство `.current` которого инициализируется переданным аргументом. Возвращенный объект сохраняется на протяжении всего времени жизни компонента.

```

import React, { useRef, useEffect } from 'react';

function TextInputWithFocusButton() {
  const inputEl = useRef(null);

  const onClick = () => {
    // `current` указывает на смонтированный элемент текстового поля
    inputEl.current.focus();
  };

  return (
    <>
      <input ref={inputEl} type="text" />
      <button onClick={onClick}>Фокус на поле ввода</button>
    </>
  );
}

```

```
);  
}
```

Пользовательские хуки

Пользовательские хуки позволяют извлекать логику компонента в повторно используемые функции. Имя пользовательского хука должно начинаться с "use".

```
import { useState, useEffect } from 'react';  
  
function useWindowWidth() {  
  const [width, setWidth] = useState(window.innerWidth);  
  
  useEffect(() => {  
    const handleResize = () => setWidth(window.innerWidth);  
    window.addEventListener('resize', handleResize);  
    return () => {  
      window.removeEventListener('resize', handleResize);  
    };  
  }, []);  
  
  return width;  
}  
  
function MyComponent() {  
  const width = useWindowWidth();  
  return <div>Ширина окна: {width}px</div>;  
}
```

Правила хуков

1. **Вызывайте хуки только на верхнем уровне.** Не вызывайте хуки внутри циклов, условий или вложенных функций.
2. **Вызывайте хуки только из функций React.** Вызывайте хуки из функциональных компонентов React или из пользовательских хуков.

Заключение

Хуки предоставляют более прямой API для многих задач, которые были сложны в классовых компонентах: управление состоянием, выполнение побочных эффектов, доступ к контексту и т.д. Они позволяют повторно использовать логику состояния без изменения иерархии компонентов, что делает код более читаемым и тестируемым.